CASE STUDY: E-HEALTH MAURITIUS

2516412

Syed Altaaf Mahomathoo Ghaboos

ICT 1215Y

Database Systems and Security

University of Mauritius

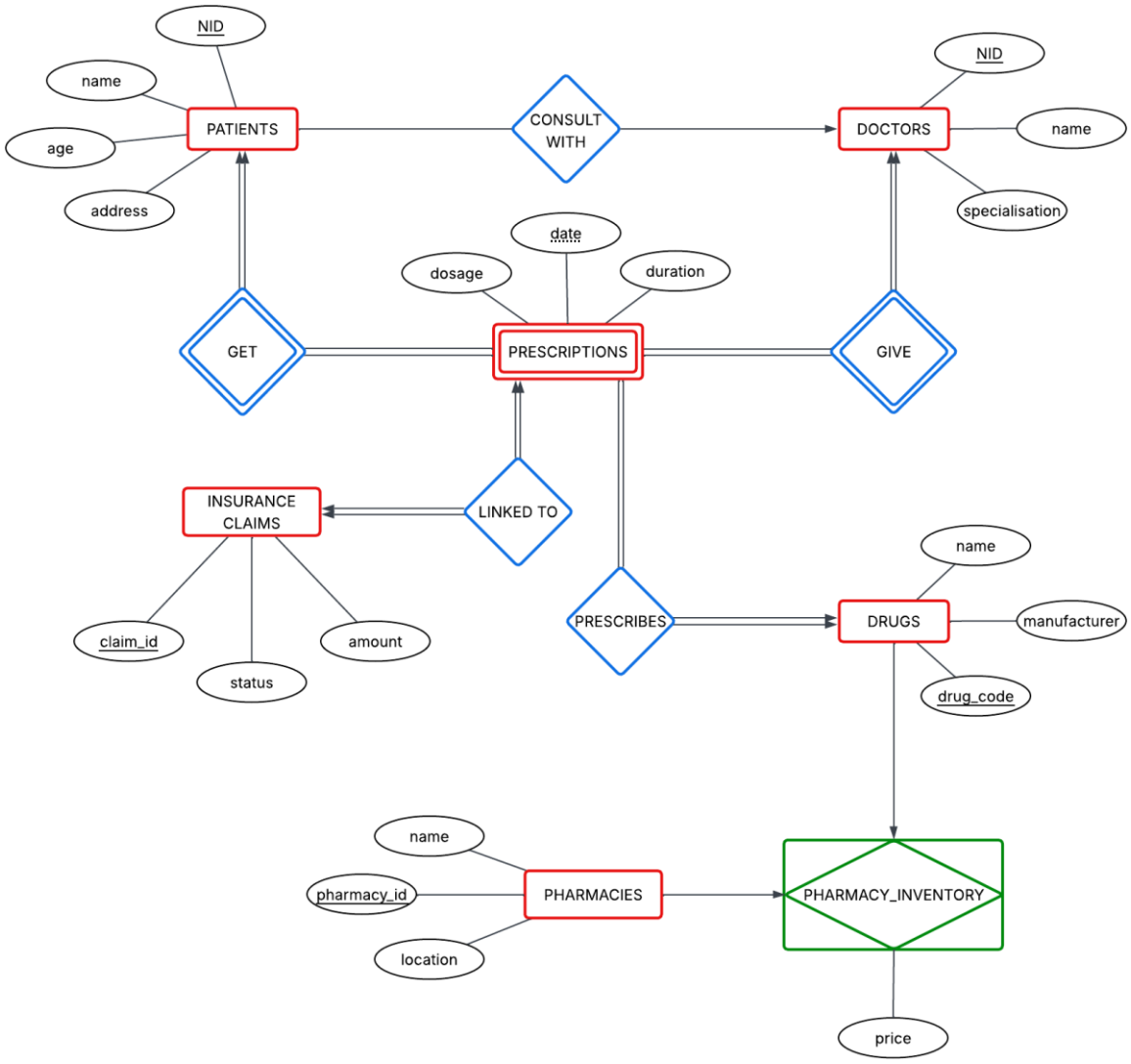
Faculty of Information, Communication and Digital Technologies

APRIL 18, 2026

Contents

(a) ERD Modelling

i. ERD for SmartHealth



Note that due to each pharmacy selling drugs at their own prices, an associative entity (in green) has been added to the ERD to remedy to that problem.

An associative entity is used to represent a *many-to-many* relationship between two entities by breaking it down into two *one-to-many* relationships. It represents the relationship between the two linked subjected entities and often can include additional attributes specific to that relationship (Athuraliya, 2026).

ii. Explain the following points:

Cardinality Choices.

The relationships of the ERD are:

- × Patients to Doctors
- × Patients to Prescriptions
- × Doctors to Prescriptions
- × Prescriptions to Insurance claims
- × Prescriptions to Drugs
- × Drugs to Pharmacies

For the PATIENTS TO DOCTORS relationship, a *many-to-one* cardinality was chosen as each patient is constrained to only have one primary doctor.

For the PATIENTS TO PRESCRIPTIONS relationship, a *one-to-many* cardinality was chosen as each patient can have multiple prescriptions, but a prescription is constrained to only belong to one patient. The relationship has total participation.

For the DOCTORS TO PRESCRIPTIONS relationship, a *one-to-many* cardinality was chosen as each doctor can give out multiple prescriptions, but a prescription is constrained to only belong to one doctor. The relationship has total participation.

For the PRESCRIPTIONS TO INSURANCE CLAIMS relationship, a *one-to-one* cardinality was chosen as each insurance claim can only be linked to one prescription only. The relationship has total participation.

For the PRESCRIPTIONS TO DRUGS relationship, a *many-to-one* cardinality was chosen as due to the limited time for the creation of the database, and because I am not versed enough in the art of SQL manipulation and database development, it was decided that prescription can be for only one drug, and a doctor will need multiple prescriptions to prescribe multiple drugs for the patient (as outlined in the constraints).

For the DRUGS TO PHARMACIES relationship, the *many-to-many* cardinality was broken into two *one-to-many* relationships via an associative entity, making it possible for multiple drugs having multiple costs at multiple pharmacies as laid out in the constraints. A *price* entity was added to the *pharmacy_inventory* associative entity to represent the price of the drugs at the pharmacies.

Assumptions made.

1. It was assumed that the *date-of-birth* of each patient is known so as to calculate the age, and so that the age is maintained and updated as needed to ensure data integrity.
2. It was assumed that each pharmacy will be maintaining their own inventory table/database, and that they will be linked to the main SmartHealth database.
3. It was assumed that no two prescriptions sharing the same *patient NID*, *doctor NID, date* and *drug_name* can be given out on any certain day. This is because those are used to make up the composite-primary key for the *PRESCRIPTIONS* table.

Alternative Modelling Proposals.

1. It was considered to directly link the *DRUGS* table and the *DOCTORS* table,
    - but the proposal was recanted as a prescription, i.e. the *PRESCRIPTION* table, should be the link between a doctor and the drugs they are prescribing.
2. It was considered to directly link the *PATIENTS* table and the *INSURANCE CLAIMS* tables, as patients are the one who would be initiating insurance claims,
    - but the proposal was recanted as they are already linked through their relations with the *PRESCRIPTIONS* tables (primary key made up of a composition involving the *patient NID*, i.e. primary key.)—it was refused as to prevent data redundancy.

(b) Relational Schema and SQL

i. Converting ERD into Relational Schema

With Primary Keys, Foreign Keys, and How Weak Entities have been handled.

Key:

- × Primary keys are underlined
- × Foreign keys are coloured
- × Foreign keys acting as primary keys are both

PATIENTS

NID	name	age	address	doctor_id

DOCTORS

NID	name	specialization

PHARMACIES

pharmacy_id	name	location

DRUGS

drug_code	name	manufacturer

PRESCRIPTIONS—the weak entity

date	dosage	duration	patient_id	doctor_id	drug_code

INSURANCE CLAIMS

claim_id	amount	status	date	patient_id	doctor_id	drug_code

PHARMACY_INVENTORY—the associative entity

pharmacy_id	drug_code	price

A weak entity, by definition, is one by which no attribute of theirs can be singled out to act as the main differentiator for the records, in other words, one that has no unequivocally distinct primary key; and for that purpose, a *discriminator* and the *primary key* of a strong entity need be combined to create the primary [composite] key for the table. A *discriminator* is an attribute of the weak entity that can act as a partial key, being the part that will differentiate the records of the weak entity; and to be combined with the *primary key* of a strong entity, that will act as the main differentiator. Without the strong entity's *primary key*, there may be duplicates in the *discriminator* that will render its job as the primary key of the table useless, with it, they form a true [composite] primary key.

For the purpose of this database, the *discriminator* in *PRESCRIPTIONS* has been chosen as the *date* entity, as it is, theoretically, the one with the least duplicates. It will be combined with *patient_id*—the value of the primary key of *PATIENTS*, with *doctor_id*—the value of the primary key of *DOCTORS*, and with *drug_code*—the value of the primary key of *DRUGS*, to make the [composite] primary key.

An associative entity, as stated before, represents the relationship between the two linked subjected entities. With each pharmacy selling the drugs at different prices, it needed to be added so as to represent that relationship. It takes in the primary keys of *PHARMACIES* and *DRUGS* and uses those as its own [composite] primary key to relate to the prices of drugs at each store.

Without the associative entity, there would be no way to store the price of each drug at each of the pharmacies. Initialising the price in the *DRUGS* entity would make all pharmacies sell the drugs at the same price, initialising the price in the *PHARMACIES* entity would mean nothing as there is no subject to relate the price to (unless all the drugs were added to the *PHARMACIES* table, which would make it too bulky and unreadable).

ii. Write SQL queries for:

Creating tables with constraints.

Before creating the tables representing the entities, the database need be created:

```
CREATE DATABASE SmartHealth;
```

To apply changes to the database, the following query is used:

```
USE SmartHealth;
```

When creating tables for the database, there is a hierarchy to be respected. Tables with no dependencies first, i.e. those with no foreign keys first. In this context, the tables concerned are: *DOCTORS*, *PHARMACIES* and *DRUGS*.

For the *DOCTORS* table:

```
CREATE TABLE doctors (
    NID VARCHAR(32) PRIMARY KEY NOT NULL,
    name NVARCHAR(128) NOT NULL,
    specialization NVARCHAR(128) NOT NULL);
```

For the *PHARMACIES* table:

```
CREATE TABLE pharmacies (
    pharmacy_id VARCHAR(32) PRIMARY KEY NOT NULL,
    name NVARCHAR(128) NOT NULL,
    location NVARCHAR(256) NOT NULL);
```

For the *DRUGS* table:

```
CREATE TABLE drugs (
    drug_code VARCHAR(32) PRIMARY KEY NOT NULL,
    name NVARCHAR(128) NOT NULL,
    manufacturer NVARCHAR(128) NOT NULL);
```

Next are the tables with dependencies from the independent tables only, in this case, *PATIENTS* and *PHARMACY_INVENTORY*.

For the *PATIENTS* table:

```
CREATE TABLE patients (
    NID VARCHAR(32) PRIMARY KEY NOT NULL,
    name NVARCHAR(128) NOT NULL,
    age INT NOT NULL CHECK (age >= 0),
    address NVARCHAR(256) NOT NULL,
    doctor_id VARCHAR(32) NOT NULL,

    FOREIGN KEY (doctor_id) REFERENCES doctors(NID));
```

For the *PHARMACY_INVENTORY* table:

```
CREATE TABLE pharmacy_inventory (
    pharmacy_id VARCHAR(32) NOT NULL,
    drug_code VARCHAR(32) NOT NULL,
    price DECIMAL(8, 2) NOT NULL CHECK (price > 0),

    PRIMARY KEY (pharmacy_id, drug_code),

    FOREIGN KEY (pharmacy_id) REFERENCES
    pharmacies(pharmacy_id),
    FOREIGN KEY (drug_code) REFERENCES
    drugs(drug_code));
```

Next comes the tables dependent on the tables that are themselves dependent on the independent tables; in this case, *PRESCRIPTIONS.*

For *PRESCRIPTIONS* table:

```
CREATE TABLE prescriptions (
    date DATE NOT NULL,
    dosage NVARCHAR(256) NOT NULL,
    duration VARCHAR(32) NOT NULL,
    patient_id VARCHAR(32) NOT NULL,
    doctor_id VARCHAR(32) NOT NULL,
    drug_code VARCHAR(32) NOT NULL,

    PRIMARY KEY(date, patient_id, doctor_id, drug_code),

    FOREIGN KEY (patient_id) REFERENCES patients(NID),
    FOREIGN KEY (doctor_id) REFERENCES doctors(NID),
    FOREIGN KEY (drug_code) REFERENCES
    drugs(drug_code));
```

Lastly in the hierarchy comes *INSURANCE_CLAIMS* as it is dependent on the *PRESCRIPTIONS* table.

For *INSURANCE_CLAIMS* table:

```
CREATE TABLE insurance_claims (
    claim_id VARCHAR(32) PRIMARY KEY NOT NULL,
    amount DECIMAL(16, 2) NOT NULL CHECK (amount > 0),

    status VARCHAR(32) NOT NULL
        CONSTRAINT claim_df DEFAULT 'in progress'
        CONSTRAINT claim_ck CHECK (status IN ('in
        progress', 'accepted', 'rejected')),

    date DATE NOT NULL,
    patient_id VARCHAR(32) NOT NULL,
    doctor_id VARCHAR(32) NOT NULL,
    drug_code VARCHAR(32) NOT NULL,

    FOREIGN KEY (date, patient_id, doctor_id, drug_code)
    REFERENCES prescriptions(date, patient_id,
    doctor_id, drug_code));
```

Populating the tables with randomly made-up data (5 entries maximum per table)

As for the previous step, the hierarchy needs to be respected; because a table cannot reference an empty field, this is *preposterous* (as would say 'Captain' Jack Sparrow).

First, the independent tables need be populated; *DOCTORS, PHARMACIES* and *DRUGS*.

Populating the *DOCTORS* table:

```
INSERT INTO doctors (NID, name, specialization) VALUES
     ('6501', 'Dr. Louis-Alexandre Berthier',
     'Cardiology'),
     ('6802', 'Dr. Louis-Nicolas Davout', 'General
     Practice'),
     ('6603', 'Dr. Michel Ney', 'General Practice'),
     ('6504', 'Dr. André Masséna', 'Dermatology'),
     ('6605', 'Dr. Jean Lannes', 'Orthopedics');
```

Populating the *PHARMACIES* table:

```
INSERT INTO pharmacies (pharmacy_id, name, location)
VALUES
     ('4601', 'HealthPlus Port Louis', 'Sir Seewoosagur
     Ramgoolam St, Port Louis'),
     ('4902', 'The Cross Curepipe', 'Royal Road,
     Curepipe'),
     ('4403', 'Pharmacie des deux frères', 'Rue des
     plaines, Mahebourg'),
     ('4404', 'Pharmacie Blue Bay', 'Blue Bay Road,
     Mahebourg'),
     ('4105', 'Das Farma', 'Sunset Boulevard, Curepipe');
```

Populating the *DRUGS* table:

```
INSERT INTO drugs (drug_code, name, manufacturer) VALUES
     ('2201', 'Paracetamol', 'Panadol'),
     ('2202', 'Ibuprofen', 'Avdil'),
     ('2403', 'Paracetamol', 'Tylenol'),
     ('2604', 'Paracetamol', 'Doliprane'),
     ('2805', 'Aspirin', 'Bayer');
```

Next, those dependent on these independent tables; *PATIENTS* and *PHARMACY_INVENTORY*.

Populating the *PATIENTS* table:

```
INSERT INTO patients (NID, name, age, address, doctor_id)
VALUES
     ('1001', 'Alexandre Macedon', 28, 'Mahebourg',
     '6501'),
     ('1802', 'Caius Camillius', 34, 'Vacoas', '6501'),
     ('1703', 'Marc Aurel', 45, 'Rose Hill', '6802'),
     ('1304', 'Bohort Gaunes', 22, 'Phoenix', '6603'),
     ('1705', 'Cleopatre Philopator', 20, 'Tamarin',
     '6605');
```

Populating the *PHARMACY_INVENTORY* table:

```
INSERT INTO pharmacy_inventory (pharmacy_id, drug_code,
price) VALUES
     ('4601', '2201', 50.00),
     ('4902', '2201', 55.00),
     ('4601', '2202', 200.00),
     ('4403', '2403', 120.00),
     ('4105', '2805', 450.00);
```

Then, the tables dependent on the dependent tables; *PRESCRIPTIONS*.

Populating the *PRESCRIPTIONS* table:

```
INSERT INTO prescriptions (date, dosage, duration,
patient_id, doctor_id, drug_code) VALUES
     ('2025-04-10', 'after each meal', '5 days', '1001',
     '6501', '2201'),
     ('2025-04-12', 'before each meal', '10 days',
     '1802', '6501', '2202'),
     ('2025-04-15', 'in the morning', '30 days', '1703',
     '6802', '2403'),
     ('2025-04-18', 'morning and afternoon', '7 days',
     '1304', '6603', '2604'),
     ('2025-04-18', 'before sleep', '14 days', '1705',
     '6605', '2805');
```

Lastly in the hierarchy is *INSURANCE CLAIMS*.

Populating the *INSURANCE_CLAIMS* table:

```
INSERT INTO insurance_claims (claim_id, amount, status,
date, patient_id, doctor_id, drug_code) VALUES
     ('9001', 1500.00, 'accepted', '2025-04-10', '1001',
     '6501', '2201'),
     ('9102', 850.00, 'in progress', '2025-04-12',
     '1802', '6501', '2202'),
     ('9903', 2200.00, 'rejected', '2025-04-15', '1703',
     '6802', '2403'),
     ('9604', 3100.00, 'accepted', '2025-04-18', '1304',
     '6603', '2604'),
     ('9905', 400.00, 'in progress', '2025-04-18',
     '1705', '6605', '2805');
```

Retrieving patients treated in 2025

```
SELECT DISTINCT
     pa.NID, pa.name, pa.age, pa.address
FROM patients pa

JOIN prescriptions pr ON pa.NID = pr.patient_id
     WHERE YEAR(pr.date) = 2025;
```

Listing pharmacies selling a specific drug (e.g., "Paracetamol")

```
SELECT
     ph.name AS PharmacyName,
     ph.location,
     ph_inv.price
FROM pharmacies ph

JOIN pharmacy_inventory ph_inv ON ph.pharmacy_id =
ph_inv.pharmacy_id

JOIN drugs d ON ph_inv.drug_code = d.drug_code

WHERE d.name = 'Paracetamol';
```

Calculating average drug price per pharmacy.

```
SELECT
     ph.name AS PharmacyName,
     AVG(ph_inv.price) AS AveragePrice
FROM pharmacies ph

JOIN pharmacy_inventory ph_inv ON ph.pharmacy_id =
ph_inv.pharmacy_id

GROUP BY ph.name;
```

Updating drug price by +5% for a given manufacturer

```
UPDATE pharmacy_inventory

SET price = price * 1.05
   WHERE drug_code IN (
        SELECT drug_code
        FROM drugs
        WHERE manufacturer = 'Doliprane');
```

(c) Database Security Analysis

i. How the System can be compromised?

COMPROMISE 1:

As is the case with any application program making use of SQL, an SQL Injection attack is a real threat that needs to be dealt with. An SQL Injection attack is one where malicious query code is dropped onto the database. This usually happens when the application accepts user-input in the form of text, a malevolent actor is then able to, through clever syntax, to 'inject' their own SQL queries whilst ignoring the legitimate code that the application is supposed to run.

COMPROMISE 2:

For the time being, no user authentication (authN) or authorization (authZ) has been put in place; meaning that anyone with access to the server can manipulate the database with impunity.

> AuthN is the process of legitimizing the credentials of the user, i.e. the system asking if the user is a legitimate one or not.
>
> AuthZ is the process of checking if the authorized user has the correct privileges to access and manipulate the subjected database.

COMPROMISE 3:

As well as neither authN nor authZ having been implemented, user classes should be created for real-world implementation, otherwise sensitive [private] data would be accessible by anyone and everyone.

COMPROMISE 4:

Data, whether at rest or in transit, need be encrypted to protect the confidentiality of those present in the database. Without this precautionary measure, everything is saved in *plain-text*, and any malicious actor vying to use these private records for ill-advised ends will be able to do so.

COMPROMISE 5:

Lastly, with the lack of auditing, the principle of non-repudiation fails. Non-repudiation means that there is accountability, that no principal parties can deny any actions undertaken by these same parties; this makes everyone responsible for the changes they make to the database, meaning no malicious actor can implement changes or data migration without the action being traced back to them.

ii. Real-World Consequences in Healthcare Context.

According to the Data Protection Act 2017, the premier regulation concerning the handling of personal data in Mauritius, any and all organization operating in the Mauritian territory, and dealing with personal data are required to implement technical, administrative and organizational measures in assurance of data security; the non-adherence to which is a criminal act under the Mauritian law.

Furthermore, taking in the security compromises outlined in the previous section, the consequences of which are as follows:

COMPROMISE 1:

> The major consequence of being subject to an SQL Injection attack is the uncertainty; not protecting yourself against it opens the door to attackers treating your database like their personal playground. Anything that can be done using *Database Definition Language* and *Database Manipulation Language* is now possible with the code injection.

COMPROMISE 2:

> With no authN and authZ, like outlined before, any and everyone with access to the server will be able to access the database (if they know what they're looking for), and from then on, the possibilities are endless. With no user filtration, any non-legitimate user is free to peruse and manipulate the database.

COMPROMISE 3:

> Having no user classes (or user roles) means that the database akin to an *open book* for all its users. Sensitive [private] data that are only meant to be viewed and manipulated by specific users will be available to all users, breaking the confidentiality principle of data security.

COMPROMISE 4:

> Not encrypting the data leaves the organization open to malicious actors having full on *view* access to the database. An unencrypted database is also more prone to malicious data manipulation as the data is stored in *plain-text*.

COMPROMISE 5:

> With no auditing, the actions of individuals on the database aren't stored, meaning that no malicious actor (be it an inside or outside user) can be held accountable and no malicious action can be traced back to its source.

iii. Mitigation Techniques.

MITIGATION TECHNIQUE 1

Against SQL Injection, preventative measures like input validation need be put in place. Input Validation is filtering user inputs through a *white* [*black*] list of allowed [disallowed] inputs before allowing processing.

MITIGATION TECHNIQUE 1

Encryption of the database, or at least of the fields containing sensitive [personal] information is paramount in the aim of protecting the subjects' privacy.

MITIGATION TECHNIQUE 1

Collection of logs (Audits), whilst not an *active* protectionary measure, makes the data protection process all the more possible; as any actions that is applied to the database gets recorded, making it easy to trace it back to its source.

iv. Comparison of Prepared Statements, Input Validation and Role-based Access Control.

Comparison:

Prepared Statements are somewhat akin to *functions* in programming, where commands/code are written in a template of SQL query, making for the set of queries executable by only invoking the prepared statement.

Input Validation, as outlined in the previous section, is comparing inputs to the database against a set of allowed and disallowed queries, and filtering them as such.

Role-based Access Control is pretty self-explanatory in its name. It involves the creation of many a set of user [role] classes, each class with its own sets of privileges attached to them.

In a security context, prepared statements primarily against SQL injection attacks, preventing queries to be dropped in by treating the user input as strictly data to be inserted into the *template* rather than code; input validation checks the user input and determines whether the query is malicious or not based on the library on inputs at its disposal; and role-based access control prevents guest users [outside of the database team], or users outside of the proper groups for that matter, from manipulating the database because they won't be *anointed* with the proper privileges.

Ranking in terms of effectiveness:

In terms of effectiveness, whilst each of these solutions work together for better security, they each have their shortcomings.

RBAC is not immune to privilege-creep or to the user account being compromised; there are ways to circumvent input validation through clever code writing or with new attack patterns if a *black* list is used, whilst, as is the case with prepared statements, if a *white* list is used, it mostly hinders the *database* team—when there are new tasks to be done with the database and queries based on that are not included in the *white* list, or prepared statements requiring a lot of planning and complicating the maintenance process and not being *storage*-friendly.

In terms of ranking, and only considering the security aspects;

1. Prepared Statements
   flawless against injection attacks
2. RBAC
   only way for outside actors to compromise the system is to compromise an internal user.
3. Input Validation
   is more of a safety net rather than the primary defense

(d) Solution Review

i. What was the most difficult design decision?

With the specifications given, storing the price of drugs for every pharmacy posed a real conundrum as I do not recall encountering this type of problem when learning DDL. Admittedly, this may be my own fault for not practicing enough, but through research I discovered the concept of *associative entity* which allowed me to moved pass this.

ii. Where could the design fail in real-world deployment?

A huge omission is that of the day-of-birth of patients, because as it stands, patients' ages need be entered manually, which makes updating their ages year-on-year nigh on impossible, especially for a large database.

Another shortcoming is having the doctors only being able to prescribe one drug at a time per prescription, meaning that patients needing multiple types of medicines need *log* around *boat-loads* of prescriptions for their ailments.

Lastly, using solely the long numerical codes as links between the entities is asking for mis-attribution of drugs, patients and doctors by a tired and overworked office worker. Numerical codes can be confusing and are not very human-readable.

iii. How to improve the system?

Where possible, the foreign keys should also include the *English* names of doctors, patients, pharmacies and drugs for a more *human* design. Makes it more readable and better for database maintenance, even if it is a redundant design.

A field for the *d.o.b* of patients would have been included, making age a calculated factor rather than a numerical text input. This is much better for database maintenance.

A prescription_id field would be added to the *PRESCRIPTIONS* table, making it not only a strong entity, but it would have a cascading effect of enabling doctors to an individual patient the same drug multiple times per day as the key wouldn't hinge on the prescription date anymore.

A system to allow doctors to prescribe multiple drugs in the same prescription would have been put in place.

(e) Transaction and Recovery

<T1 start>
<T1, A, 100, 120>
<T2 start>
<T2, B, 200, 250>
<T1 commit>
<CHECKPOINT {T2}>
<T3 start>
<T3, A, 120, 150>
CRASH

i. Identify Committed Transactions and Uncommitted Transactions

T1 is a committed transaction, as seen by the *commit* flag. The instructions of T1 are thus permanent in the database.

T2 and T3 are uncommitted transactions, as they have no *commit* flags. None of their instructions have been saved.

ii. Perform Recovery using Deferred Update and Immediate Update

A deferred update is one where changes are not stored to the database before manually doing so (using a *commit* flag), and an immediate update saves all changes as they are done.

DEFERRED UPDATE RECOVERY

T1 has already been saved on disk since it was committed (delay from log to writing onto the database disk ignored as evidence by the checkpoint after the commit). No action needed.

T2 and T3 are ignored as they haven't been committed to the database yet.

IMMEDIATE UPDATE RECOVERY

T1 is skipped as even though it was committed, no REDO needed as it happened before the checkpoint—it is confirmed to be written on the disk.

UNDO T2 and T3 as their corrupted data have been written onto the database storage as they both happened after the checkpoint (the last confirmed *clean* state of the disk). T2 need be undone because it was not explicitly committed, even though the update happened before the checkpoint.

iii. Provide final values for A and B.

A = 120, B = 200

For both approaches.

iv. Explain why both approaches may or may not produce the same result.

They produced the same result as only T1 was explicitly committed before the checkpoint; even though T2 was updated, it was active after the checkpoint and has not been explicitly committed, and thus immediate update recovery cannot accept the new value as correct.

v. Which approach is more suitable for Healthcare Systems and why?

Deferred update is more suitable as it ensures only explicitly committed changes are written in the disk; little chance of corrupted/incorrect data in the files meaning that it is safer for the patients to follow the prescription even if it was affected by a crash.

(f) References

Athuraliya, A. (2026) *Easy guide to Chen notation for entity-relationship diagrams*. Creately. Available at: https://creately.com/guides/chen-notation-in-erd/ (Accessed: 11 April 2026).

Mauritius (2017) *Data Protection Act 2017*. Port Louis: Government of Mauritius. Available at: https://dataprotection.govmu.org/Documents/DPA_2017.pdf (Accessed: 19 April 2026).